Fork–exec

From Wikipedia, the free encyclopedia

July 16: The internet we were promised

We're sorry we've asked you a few times recently, but it's Wednesday, July 16 – please don't wait until tomorrow to help. We're happy you consult Wikipedia often. If just 2% of our most loyal readers gave $2.75 today, we'd reach our goal quickly. Most readers don't donate, so if Wikipedia has given you $2.75 of knowledge, please give. Any contribution helps, whether it's $2.75 one time or monthly.

Give $2.75
Give a different amount
WIKIMEDIA FOUNDATION

MAYBE LATER I ALREADY DONATED CLOSE

This article includes a list of general references, but **it lacks sufficient corresponding inline citations**. Please help to improve this article by introducing more precise citations. *(March 2022)* (*Learn how and when to remove this message*)

Fork–exec is a commonly used technique in Unix whereby an executing process spawns a new program.

Description [edit]

`fork()` is the name of the system call that the parent process uses to "divide" itself ("fork") into two identical processes. After calling `fork()`, the created child process is an exact copy of the parent except for the return value of the fork() call. This includes open files, register state, and all memory allocations, which includes the program's executable code. In some cases the two continue to run the same binary, but often one (usually the child) switches to running another binary executable using the `exec()` system call.

When a process forks, a complete copy of the executing program is made into the new process. This new process is a child of the parent process, and has a new process identifier (PID). The `fork()` function returns the child's PID to the parent process. The `fork()` function returns 0 to the child process. This enables the two otherwise identical processes to distinguish themselves from each other.

The parent process can either continue execution or wait for the child process to complete. The child, after discovering that it is the child, will most often then replace itself completely with another program, so that the code and address space of the original program are lost. This replacement is, however, a choice of the architecture one builds the given program on, and is therefore not an obligatory step in the child process' life.

If the parent chooses to wait for the child to die, then the parent will receive the exit code of the program that the child executed. To prevent the child becoming a zombie the parent should call wait on its children, either periodically or upon receiving the SIGCHLD signal, which indicates a child process has terminated.

One can also asynchronously wait on their children to finish, by using a signal handler for SIGCHLD, if they need to ensure everything is cleaned up. Here's an example of a signal handler that catches any incoming SIGCHLD signals and handles multiple concurrent signals received.

```
void cleanup(int signal) {
  while (waitpid((pid_t) (-1), 0, WNOHANG) > 0) {}
}
```

When the child process calls `exec()`, all data in the original program is lost, and it is replaced with a running copy of the new program. This is known as overlaying. Although all data are replaced, the file descriptors that were open in the parent are closed only if the program has explicitly marked them *close-on-exec*. This allows for the common practice of the parent creating a pipe prior to calling `fork()` and using it to communicate with the executed program.

Microsoft Windows does not support the fork–exec model, as it does not have a system call analogous to `fork()`. The `spawn()` family of functions declared in process.h can replace it in cases where the call to `fork()` is followed directly by `exec()`.

> When a fork syscall is made on WSL, lxss.sys does some of the initial work to prepare for copying the process. It then calls internal NT APIs to create the process with the correct semantics and create a thread in the process with an identical register context. Finally, it does some additional work to complete copying the process and resumes the new process so it can begin executing.
>
> —Jack Hammons of Microsoft[1]

References [edit]

1. ^ "WSL System Calls". 8 June 2016.

- "File descriptors across fork(2)/exec(2)", Operating Systems (Course 304-427B), Franco Callari, Electrical Engineering Department, McGill University
- "fork and exec", Tim Love, University of Cambridge Engineering Department
- *Advanced Programming in the UNIX Environment*, W. Richard Stevens, Addison-Wesley ISBN 0-201-56317-7
- *Unix Power Tools*, Jerry Peek, Tim O'Reilly, Mike Loukides, O'Reilly, ISBN 1-56592-260-3

Categories: C POSIX library | Process (computing)

This page was last edited on 5 December 2024, at 18:38 (UTC).

Text is available under the Creative Commons Attribution-ShareAlike 4.0 License; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.

Privacy policy About Wikipedia Disclaimers Contact Wikipedia Code of Conduct Developers Statistics Cookie statement Mobile view

a WIKIMEDIA project
Powered by MediaWiki

Sorry to interrupt, but our fundraiser won't last long.
This Wednesday, we ask you to join the 2% of readers who give. If everyone reading this right now gave just $2.75, we'd hit our goal quickly. $2.75 is all we ask.
GIVE $2.75 MAYBE LATER